

Free Form Testing Agent

Document: 01_vision.md

Version: 1.0

Status: Draft

Author: Kalpesh

Date: June 2026

1. Executive Summary

The Free Form Testing Agent (FFTA) is an autonomous software testing system capable of exploring unknown applications without predefined test cases. Unlike traditional automated testing frameworks that require manually authored test scripts, the Free Form Testing Agent uses exploration strategies, state modeling, artificial intelligence, and validation heuristics to discover application behavior, identify defects, and automatically generate reproducible test cases.

The initial implementation target is a Calculator application running on Windows. The calculator domain provides a constrained environment that enables the development and validation of core autonomous testing capabilities before expanding to larger desktop, web, mobile, and industrial software systems.

The long-term vision of this project is to build a generalized AI-powered testing agent capable of performing exploratory testing similarly to a human software tester.

2. Problem Statement

Traditional software testing suffers from several limitations:

- Test cases must be manually written.
- Maintenance costs increase as software evolves.
- Exploratory testing requires skilled human testers.
- Unknown application states are difficult to discover automatically.
- Edge cases are often missed.
- Regression suites become expensive to maintain.

Current AI-assisted testing solutions mostly generate tests from existing specifications rather than autonomously discovering software behavior.

The Free Form Testing Agent addresses this problem by creating an autonomous system capable of:

- Exploring applications independently.
 - Understanding application states.
 - Generating hypotheses.
 - Executing actions.
 - Detecting anomalies.
 - Reporting defects.
 - Producing reproducible test cases.
-

3. Project Vision

The vision of the Free Form Testing Agent is:

"Given any application, the system should autonomously explore, understand, test, validate, and report software defects without requiring manually written test cases."

The system should eventually perform similarly to an experienced exploratory software tester by:

- Understanding available actions.
 - Exploring unknown workflows.
 - Testing boundary conditions.
 - Detecting failures.
 - Learning from previous explorations.
 - Generating reusable automated tests.
-

4. Project Objectives

Primary Objectives

The primary objectives of the project are:

Objective 1

Create a generalized application abstraction layer.

Objective 2

Build an application state representation model.

Objective 3

Implement autonomous exploration algorithms.

Objective 4

Integrate local Large Language Models.

Objective 5

Automatically discover defects.

Objective 6

Generate reproducible test cases.

Objective 7

Support offline execution.

Secondary Objectives

Secondary objectives include:

- State graph visualization.
- Test coverage measurement.
- Screenshot comparison.
- Historical learning.
- Session replay.
- Bug prioritization.
- Test optimization.
- Multi-agent reasoning.

5. Project Scope

Phase 1 Scope

The first implementation phase focuses on:

Application

Windows Calculator

Features

- Launch application
- Observe UI state
- Identify controls
- Execute actions
- Build state graph
- Detect failures
- Generate reports

Constraints

- Offline only
 - Local LLM only
 - Single application
 - Single machine execution
-

Phase 2 Scope

Support desktop applications:

- WinForms
 - WPF
 - UWP
 - Qt
 - Electron
 - Native Windows applications
-

Phase 3 Scope

Support web applications:

- Selenium
 - Playwright
 - Browser automation
 - API interception
-

Phase 4 Scope

Support:

- Mobile applications
- REST APIs

- Games
 - Industrial software
 - Embedded software
-

6. Non-Goals

The following capabilities are outside the initial scope:

Manual test authoring tools

The system will not initially replace existing test automation frameworks.

Performance testing

Load testing and stress testing are out of scope.

Security testing

Penetration testing capabilities are excluded.

Distributed execution

Cluster execution support is not planned initially.

Cloud execution

The initial implementation is designed for local execution only.

7. Guiding Principles

The project follows several guiding principles.

Principle 1: Autonomous First

The system should operate with minimal human intervention.

Principle 2: Offline First

All reasoning and execution should be possible without internet access.

Principle 3: Extensible Architecture

Every subsystem should be replaceable.

Principle 4: Explainability

The agent must explain:

- Why an action was selected.
 - What was expected.
 - What actually happened.
 - Why something is considered a defect.
-

Principle 5: Reproducibility

Every discovered bug must be reproducible.

Principle 6: Deterministic Execution

Given the same state and configuration, the system should produce identical behavior.

8. System Capabilities

The Free Form Testing Agent will provide the following capabilities.

Application Discovery

The system can:

- Launch applications.
 - Observe windows.
 - Enumerate controls.
 - Identify interaction points.
-

State Modeling

The system can:

- Capture UI state.
 - Normalize states.
 - Hash states.
 - Compare states.
 - Build state graphs.
-

Autonomous Exploration

The system can:

- Explore workflows.
 - Discover navigation paths.
 - Search unknown states.
 - Avoid repeated exploration.
-

AI Reasoning

The system can:

- Analyze state.
 - Predict actions.
 - Generate hypotheses.
 - Select exploration strategies.
-

Validation

The system can:

- Verify outputs.
 - Detect crashes.
 - Detect freezes.
 - Detect logical errors.
 - Detect exceptions.
-

Bug Detection

The system can identify:

- Crashes
 - Exceptions
 - UI failures
 - Logical errors
 - State corruption
 - Invalid outputs
-

Test Generation

The system can generate:

- Reproduction steps
 - Gherkin scenarios
 - Reqnroll tests
 - Pytest tests
 - Regression suites
-

9. System Workflow

The expected workflow is:

```
User Goal
  ↓
Launch Application
  ↓
Observe State
  ↓
Build State Model
  ↓
Store State
  ↓
Reason About Actions
  ↓
Execute Action
  ↓
Observe Result
  ↓
Validate Result
```


↓
Detect Bug
↓
Generate Report
↓
Generate Test Cases

10. Example Execution

Example:

Initial State

Display = 0
Available Buttons:

0-9
+
-
*
/
=
C

Agent Reasoning

Hypothesis:
Division by zero may cause failure.

Plan:
Execute:

7
/
0
=

Result

Error dialog appeared.

Agent Decision

Potential defect detected.

Generated Test

Feature: Calculator

Scenario: Division by zero

Given calculator is launched
When user enters 7
And user enters /
And user enters 0
And user presses =
Then error dialog should appear

11. Success Metrics

The project will be evaluated using the following metrics.

Exploration Metrics

- Total states discovered
 - Total transitions discovered
 - Coverage percentage
 - Exploration depth
-

Testing Metrics

- Bugs discovered

- Unique bugs discovered
 - False positives
 - Reproducibility rate
-

AI Metrics

- Correct decisions
 - Useful decisions
 - Exploration efficiency
 - Reasoning accuracy
-

Test Generation Metrics

- Generated tests
 - Successful replay percentage
 - Test coverage
 - Maintenance cost
-

12. Technology Stack

Programming Language

Python 3.12

Local LLM

Ollama
Qwen3
DeepSeek-R1
Llama

Agent Framework

LangGraph

Desktop Automation

pywinauto
uiautomation
pyautogui

Database

SQLite
SQLAlchemy

Graph Engine

NetworkX

Validation

SymPy

Reporting

Jinja2
Allure

13. Long-Term Vision

The final vision of the project is:

Input:

"Test this application."

↓

Launch

↓

Understand

↓

Explore

↓

Reason

↓

Validate

↓

Find Bugs

↓

Generate Reports

↓

Generate Test Cases

↓

Generate Regression Suite

↓

Learn

↓

The Free Form Testing Agent aims to become a generalized autonomous software testing system capable of performing exploratory testing at a level comparable to experienced human software testers while maintaining reproducibility, scalability, and extensibility.

14. Conclusion

The Free Form Testing Agent represents an approach to software testing where autonomous agents replace manually authored test cases with intelligent exploration, reasoning, validation, and learning.

The Calculator application serves as the first domain for validating the core architecture, after which the system can be progressively extended toward becoming a generalized AI-powered software testing platform.